

Additional Assignment: Real-time Stock Data (Kafka / SSE)

Estimated time: ~60 minutes

Tools allowed: any AI coding assistant (Cursor AI, VS Code + Copilot, etc.) — vibe coding encouraged!

1. Context

During this lab you will consume a **live stream of fictitious stock price ticks** for 50 companies.

The data is produced by a Kafka broker running on the instructor's server and exposed via a simple **HTTP / SSE API** — no Kafka client libraries required.

2. Step 1 — Get your API key

Open in the browser:

```
https://add.piotrkojalowicz.dev/
```

1. Enter the **shared class password** (announced by the instructor).
2. Copy the generated **API key** — it is **unique per student**.
You will need it in every API call.

 The service is open only during class hours (until 19:00).
Rate limit: first 10 requests free, then max 1 request / 10 s per key.

3. Available API endpoints

Base URL: `https://add.piotrkojalowicz.dev`

Authentication

Every request must include the key as a header **or** a query param:

Method	Example
Header	<code>X-API-Key: <YOUR_KEY></code>
Query param	<code>?api_key=<YOUR_KEY></code> (convenient for browser)

GET /api/tickers

Returns the full list of 50 companies.

```
curl -H "X-API-Key: <KEY>" "https://add.piotrkojalowicz.dev/api/tickers"
```

Browser:

```
https://add.piotrkojalowicz.dev/api/tickers?api_key=<KEY>
```

GET /api/latest?ticker=ACME

Returns the most recent tick for one or more tickers.

```
curl -H "X-API-Key: <KEY>" \
  "https://add.piotrkojalowicz.dev/api/latest?ticker=ACME&ticker=ALFA"
```

Browser:

```
https://add.piotrkojalowicz.dev/api/latest?ticker=ACME&ticker=ALFA&api_key=<KEY>
```

Response format:

```
{
  "data": [
    {
      "ticker": "ACME",
```

```

    "ts": "2026-05-07T15:30:00+02:00",
    "price": 123.45,
    "currency": "PLN",
    "volume": 1200,
    "seq": 42
  }
]
}

```

Data is retained for **~10 minutes** only — older ticks are automatically deleted.

GET /api/stream?ticker=ACME

Live stream via **Server-Sent Events (SSE)** — new tick sent every ~second.

```

curl -N -H "X-API-Key: <KEY>" \
  "https://add.piotrkojalowicz.dev/api/stream?ticker=ACME"

```

Browser (live view):

```
https://add.piotrkojalowicz.dev/api/stream?ticker=ACME&api_key=<KEY>
```

Events arrive as:

```

event: tick
data: {"ticker":"ACME","ts":"...","price":124.10,...}

```

4. Task — Build 2 small apps

Use any language and framework. AI coding tools are recommended.

App #1: Realtime Dashboard

Goal: show live price updates for selected companies.

Requirements: - User can **select one or more tickers** from the list - App connects to `/api/stream` and shows **live updates** in the UI - Each update shows at minimum: ticker, price, timestamp - Optionally: short price history (last 20–30 ticks) or a mini chart

Hint: Use `EventSource` (JavaScript) or an SSE client library in your language.

App #2: History Downloader / Viewer

Goal: fetch recent data, display it, and save it locally.

Requirements: - User can **select tickers** and a **time range** (e.g. last 1–10 minutes) - App fetches data from `/api/latest` (repeatedly, with a delay ≥ 10 s) or `/api/stream` - Displays results as a **table or chart** in the UI - Allows **saving to CSV or JSON**

Remember: data older than ~10 minutes is gone. Design your polling accordingly.

5. Deliverables

Submit (e.g. as a GitHub repo or ZIP):

- [] Source code for both apps
- [] `README.md` describing:
 - how to run locally
 - how the API key is configured (env variable, `.env` file, etc.)
 - which endpoints are used and how
- [] The API key must **not** be hardcoded in the repository

6. Grading

See `ACCEPTANCE.md` for the full checklist.

Criterion	Weight
App #1 works (live stream, ticker selection, UI)	40%
App #2 works (data fetch, time range, save to file)	40%

Criterion	Weight
Code quality & README	20%

7. List of tickers

Ticker	Company
ACME	ACME Corp.
ALFA	Alfa Technologies
BETA	Beta Retail Group
CASH	CashBank
CLOUD	CloudNine
COAL	Coal Energy
COPR	Copper Mining Co.
DATA	DataWorks
DEVS	DevStudio
DRON	Dronix
ECO	EcoPower
EDU	EduNext
ENRG	Energo
FARM	FarmFoods
FINX	FinX
FOOD	FoodBox
FUEL	FuelOne
GAME	GameForge
GRIN	Green Invest

Ticker	Company
HEAL	HealTech
HOME	HomeBuild
HYPE	Hype Media
INSR	InsureCo
IOT	IoT Systems
JET	Jet Logistics
LABS	Labs Research
LEND	Lendify
LOGI	LogiWare
MALL	Mall Retail
MEDI	MediCare
META	MetaCom
MOBI	MobiTel
MOVE	MoveNow
NET	Netlink
NOVA	Nova Ventures
OILS	OilSands
PARK	Park Realty
PHAR	Pharmax
PLNT	Plantio
PROD	Prodigo
QBIT	QBit Quantum
RAIL	Rail Cargo
ROBO	RoboMakers

Ticker	Company
SAFE	SafeSecurity
SHIP	ShipIt
SHOP	ShopNow
SOLR	Solaris
TEL	TelcoPlus
TRVL	TravelBee
WATR	WaterWorks

8. Quick-start example (Python)

```
import requests, os

API_KEY = os.environ["ADD_API_KEY"] # set in .env or shell
BASE    = "https://add.piotrkojalowicz.dev"
HEADERS = {"X-API-Key": API_KEY}

# list tickers
tickers = requests.get(f"{BASE}/api/tickers", headers=HEADERS).json()
print(tickers["tickers"][:5])

# latest price for ACME
data = requests.get(f"{BASE}/api/latest", params={"ticker": "ACME"},
headers=HEADERS).json()
print(data)
```

SSE stream (Python):

```
import sseclient, requests, os

API_KEY = os.environ["ADD_API_KEY"]
resp = requests.get(
    "https://add.piotrkojalowicz.dev/api/stream",
    params={"ticker": "ACME"},
    headers={"X-API-Key": API_KEY},
    stream=True,
```

```
)  
for event in sseclient.SSEClient(resp):  
    if event.event == "tick":  
        print(event.data)
```

JavaScript (browser / Node):

```
const key = "YOUR_KEY";  
const es = new EventSource(  
    `https://add.piotrkojalowicz.dev/api/stream?ticker=ACME&api_key=${key}`  
);  
es.addEventListener("tick", e => console.log(JSON.parse(e.data)));
```